Data re-uploading in Quantum Machine Learning for time series: application to traffic forecasting

Nikolaos Schetakis[1,2,*], Paolo Bonfini[3], Negin Alisoltani[4], Konstantinos Blazakis[5], Symeon I. Tsintzos[6], Alexis Askitopoulos[6], Davit Aghamalyan[7], Panagiotis Fafoutellis[8], and Eleni I. Vlahogianni[8]

[1]Computational Mechanics and Optimization Laboratory, School of Production Engineering and Management, Technical University of Crete, 73100 Chania, Greece
[2]Quantum Innovation Pc, 73100 Chania, Greece
[3]Alma-Sistemi Srl, 00012 Guidonia, Italy
[4]Univ Gustave Eiffel, COSYS, GRETTIA, Paris, F-77454, France
[5]School of Electrical and Computer Engineering, Technical University of Crete, 73100, Greece
[6]QUBITECH Pc, Athens, 15231, Greece
[7]Institute of High Performance Computing, A*STAR (Agency for Science, Technology and Research), 1 Fusionopolis Way, #16-16 Connexis, Singapore 138632
[8]Department of Transportation Planning and Engineering, School of Civil Engineering, National Technical University of Athens, 5 Iroon Polytechniou Str, Zografou Campus, 157 73 Athens, Greece
[*]nischetakis@tuc.gr

arXiv:2501.12776v1 [quant-ph] 22 Jan 2025

ABSTRACT

Accurate traffic forecasting plays a crucial role in modern Intelligent Transportation Systems (ITS), as it enables real-time traffic flow management, reduces congestion, and improves the overall efficiency of urban transportation networks. With the rise of Quantum Machine Learning (QML), it has emerged a new paradigm possessing the potential to enhance predictive capabilities beyond what classical machine learning models can achieve. In the present work we pursue a heuristic approach to explore the potential of QML, and focus on a specific transport issue. In particular, as a case study we investigate a traffic forecast task for a major urban area in Athens (Greece), for which we possess high-resolution data. In this endeavor we explore the application of Quantum Neural Networks (QNN), and, notably, we present the first application of quantum data re-uploading in the context of transport forecasting. This technique allows quantum models to better capture complex patterns, such as traffic dynamics, by repeatedly encoding classical data into a quantum state. Aside from providing a prediction model, we spend considerable effort in comparing the performance of our hybrid quantum-classical neural networks with classical deep learning approaches. Our results show that hybrid models achieve competitive accuracy with state-of-the-art classical methods, especially when the number of qubits and re-uploading blocks is increased. While the classical models demonstrate lower computational demands, we provide evidence that increasing the complexity of the quantum model improves predictive accuracy. These findings indicate that QML techniques, and specifically the data re-uploading approach, hold promise for advancing traffic forecasting models and could be instrumental in addressing challenges inherent in ITS environments.

1 Introduction

Traffic forecasting is a crucial component of modern urban transportation systems, directly influencing traffic management, congestion control, and the efficiency of ITS[1,2]. Accurate traffic predictions enable timely interventions to prevent traffic congestion, optimize traffic signal timing, and improve road utilization[3–6]. The increasing availability of high-resolution traffic data generated from sensors, connected vehicles, and other sources has led to the widespread adoption of Deep Learning (DL) models such as Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs) to address this complex forecasting task[7,8]. These models have shown significant improvements in accuracy over traditional statistical and machine learning methods but come with limitations, particularly in computational efficiency and the ability to explain intricate relationships within the data[9].

A key challenge associated with these DL models lies in their computational demands, which include long training times, high complexity, and difficulties in capturing both the temporal and spatial dependencies inherent in traffic data. As transportation networks grow increasingly complex and the need for real-time forecasting becomes more critical, the search for more powerful and efficient alternatives to classical DL models intensifies. This raises an important question: can emerging quantum technologies help overcome these limitations?

In the following section, we explore the potential advantages offered by these quantum approaches within the current era of quantum devices, known as the "Noisy Intermediate-Scale Quantum" (NISQ)[10,11] era. This era is characterized by quantum processors with a moderate number of qubits (50–500), which, while promising, are still prone to noise and lack scalability, robust error correction, and the capability for full-scale quantum advantage.

In the last few years, tremendous advancements have reshaped the NISQ landscape. NISQ devices have provided a glimpse into the potential of quantum systems to outperform classical computers in specific tasks. One of the landmark achievements in this domain was Google's demonstration of "quantum supremacy" using their Sycamore chip[12]. The term "quantum supremacy"[13] refers to the ability of a quantum computer to perform a computation that is exponentially faster than any classical computer. However, Google's demonstration, while a significant milestone, was critiqued for not providing a practical advantage, as the specific task performed did not have direct real-world applications. Another significant experiment in the realm of quantum advantage was conducted using the Jiuzhang photonic quantum computer[14]. The quantum advantage was demonstrated in the complexity of sampling from a Torontonian matrix, which scales exponentially with the number of output photon clicks. Despite showing a quantum advantage, the Jiuzhang experiment did not achieve quantum supremacy because the photonic quantum computer used was not programmable.

It is truly breathtaking how rapid our experimental progress is in paving the way toward fault-tolerant error correction. On this journey first, a pioneering contribution arrived in 2022 from M. Lukin's group[15], where they utilized a 2-dimensional controllable atomic array of 256 Rydberg atoms in order to achieve faithful error mitigation. This experiment equipped the community with the confidence that we might not be that far behind to demonstrate an example in the current quantum computers that error correction can work. Such a demonstration would imply the error threshold theorem, which in turn implies that with an increasing number of qubits, errors decrease in the system. With this kind of target in mind, researchers at Google pioneered its Willow 100 qubit quantum chip[16] and recently achieved error correction below the error threshold. Given this experimental progress, which has happened in just the last few years, one might argue that we are slowly entering into the *post*-NISQ era, where, by building capabilities in error-correction, error-mitigation, and dynamical noise suppression, we will eventually lead to the arrival of the fully *fault-tolerant* era. Thus, the current quantum computing era might be more appropriately called the "soft post-NISQ" era.

It has been well conceived by several experts in the field that one of the most promising areas of research to obtain practical advantage is Quantum Machine Learning (QML)[17–19], which has emerged through the cross-fertilization of ideas and methodologies between Quantum Computing and Classical Machine Learning. QML has indeed become a fascinating research area, offering potential quantum speedups for various computational tasks. For example, the HHL algorithm[20], named after its developers Harrow, Hassidim, and Lloyd, is a well-known quantum algorithm for solving linear systems of equations and is often cited as an example of quantum speedup. It provides exponential speedup in specific scenarios compared to classical algorithms, particularly when dealing with sparse matrices or when only some properties of the solution are required[21]. Due to the growing interest in this topic, the latest developments are now covered in recent surveys[22]. Similarly, quantum algorithms for tasks like finding eigenvectors and eigenvalues[23], as well as performing principal component analysis (PCA)[24], have shown promise in achieving speedups over classical methods.

These algorithms use quantum parallelism and other quantum properties to process information more efficiently, establishing QML as a promising new paradigm[25]. Basically, quantum computing offers distinct advantages over classical computing in addressing complex optimization and machine learning tasks[26]. By exploiting the parallelism enabled by quantum effects such as entanglement and superposition, quantum algorithms can process vast datasets simultaneously, potentially providing exponential speedup for specific computations. This capability can significantly enhance the performance of machine learning models by efficiently exploring large solution spaces. For instance, one can train the quantum circuit to achieve maximal separation between the data clusters in the Hilbert space, paving the way for the development of robust quantum classifiers. Quantum computing's ability to handle high-dimensional data and capture complex patterns makes it particularly suitable for applications such as traffic forecasting, where intricate relationships within the data must be captured. To accelerate progress in QML, exploring heuristic algorithms, such as those employed in this study, is essential. Although these algorithms currently lack formal theoretical backing, they have demonstrated effectiveness in certain problem domains through cross-disciplinary insights and domain expertise.

Additionally, novel techniques such as data re-uploading[27], which involves encoding classical data into quantum circuits multiple times, can significantly enhance the expressivity of quantum models without drastically increasing computational demands. This technique holds great promise for the future capabilities of QML. Data re-uploading is achieved by catenating repeating units in sequence. Single-qubit rotations applied multiple times within the circuit generate the necessary non-linearity to construct a functional neural network. A quantum circuit can then be organized as a series of data re-uploading and single-qubit processing units. Moreover, it has been demonstrated that a single qubit data-reuploading circuit can be utilised both as being a universal quantum classifier[28] and being a universal approximant[29]. Several recent studies indicate that data re-uploading can positively impact the performance and trainability of quantum models.

In this paper, we propose the application of Quantum Machine Learning (QML), including data re-uploading, to the challenge of traffic forecasting. Our primary goal is to investigate whether a *hybrid* quantum-classical neural network architecture can match or outperform purely *classical* neural networks in predicting traffic flow. As a case study, we utilize high-resolution traffic data from the city of Athens, Greece (Figure 1).

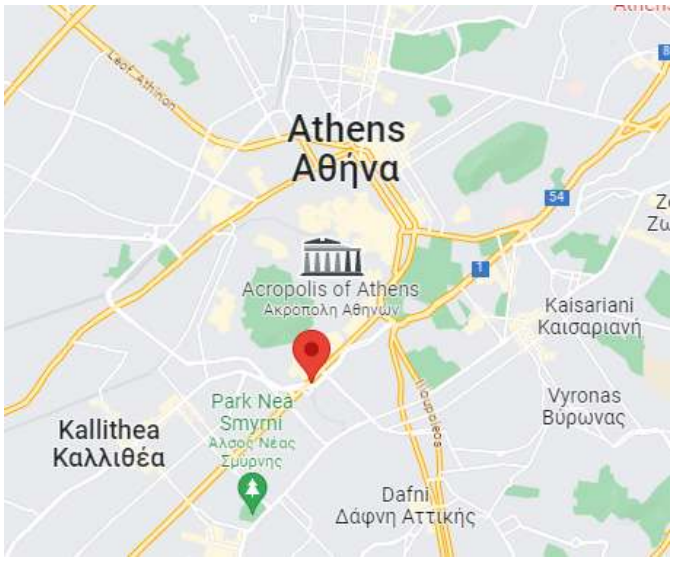

Figure 1. Location of the loop detector used in this study, relative to the center of Athens, Greece.

The core of our approach involves two distinct scenarios. In Scenario A, we focus on fully connected neural networks (NNs), replacing a classical fully connected layer with a quantum layer. This scenario allows us to investigate whether quantum layers can encode information more efficiently and capture the underlying traffic flow patterns better than classical counterparts.

Scenario B explores recurrent networks, where we integrate quantum layers with data re-uploading. This technique mimics the recursive structure of LSTMs by re-uploading input data multiple times into the quantum circuit. This allows us to assess the capability of quantum layers to model time series data by capturing both short- and long-term dependencies within the traffic data.

We compare the performance of these hybrid models with their fully classical counterparts in terms of forecasting accuracy and computational efficiency. The models are trained and tested on real-world traffic data with a time resolution of 1.5 minutes, covering 40 days of traffic flow collected from one of Athens' busiest roads. To ensure a rigorous evaluation, we apply a 5-fold gap cross-validation protocol, which helps prevent temporal leakage between the training and testing sets and enhances the generalizability of our findings.

Our results demonstrate that while classical models remain highly efficient in terms of computational complexity, hybrid quantum-classical models show promise in improving forecasting accuracy, especially as the complexity of the quantum layer increases. Furthermore, we provide evidence that using quantum variational circuits in combination with data re-uploading allows the model to better capture the complex patterns and temporal dependencies in the traffic data, potentially offering a novel way forward for traffic forecasting in ITS.

The contribution of this paper is, therefore, twofold:

- We present one of the first applications of QML to traffic forecasting, demonstrating the feasibility and potential of quantum-enhanced models for real-world forecasting tasks. Moreover, to our knowledge, this is the *first study* that successfully employs data re-uploading to a traffic prediction scenario.
- We offer a detailed comparison between classic and quantum approaches, highlighting the strengths and weaknesses of each and providing insights into the conditions under which quantum models may offer a tangible advantage.

The paper is organized as follows: Section 2 provides a comprehensive review of existing research on DL methods for traffic forecasting and explores the emerging role of quantum computing in transportation. Section 3 details our model architectures, data sources, and the hybrid quantum-classical approach we employed. In Section 4, we present our findings, including performance metrics and insights into the convergence behavior of the models. Finally, Section 5 discusses the implications of our results and outlines future research directions, while Section 6 concludes the paper with a summary of key contributions.

2 Related Work

2.1 Classical Deep Neural Networks for Traffic Forecasting

Over the past few decades, significant advancements in telecommunication technology and computing systems have enabled researchers and practitioners in the field of traffic prediction to increasingly focus on DL. This natural progression has been driven by the growing availability of vast amounts of relevant data, further facilitating the adoption and utilization of DL techniques in this domain (e.g.,[30,31]). One of the primary advantages attributed to DL methods is their notable superiority in prediction accuracy compared to traditional Statistical and Machine Learning approaches[32].

Currently, in the realm of traffic conditions forecasting, Recurrent Neural Networks (RNNs) and, more specifically, LSTM networks, have emerged as the most widely adopted models. These models are not only popular but are often combined with other architectures to achieve remarkably precise predictions (e.g.,[33,34]). A drawback associated with these models is that, when dealing with long sequences, their capacity to retain information from distant-past timesteps may diminish, a phenomenon known as the "vanishing gradient issue"[1]. Furthermore, while the RNN module and its variations excel in handling time-series problems, they are limited in their ability to capture spatial relationships within traffic data (e.g.,[35]).

Conversely, CNNs, primarily employed in image recognition and computer vision tasks, are harnessed in traffic forecasting to effectively leverage spatial relationships (e.g.,[1]). In implementing CNNs, the road network is typically represented as a 2-dimensional grid, similarly to an image. However, since this representation is effectively static, a common approach is to combine CNNs with RNNs. This combination allows for the exploitation of consecutive images, enabling a better understanding and modeling of temporal dynamics in traffic forecasting (e.g.,[36,37]).

Recently, Graph Convolutional Neural Networks (GCNNs) have emerged as an alternative to CNNs. Unlike CNNs, which operate in the Euclidean domain and represent the road network as an image, GCNNs extend the convolution operation to accommodate more general graph-structured data, making them more suitable for effectively representing a road network[9]. The model's input consists of the graph-structured data's adjacency matrix, which not only reflects the nodes' connectivity but may also capture statistical correlations. Additionally, a set of features is provided for each node, such as measured traffic flow, speed, and more. GCNNs have proven to be the current state-of-the-art in traffic prediction and have been prominently employed in several recent and noteworthy research endeavors (e.g.,[38–40]).

Despite the high accuracy of Deep Neural Networks compared to previous approaches, their applicability still faces several challenges, which are outlined below.

- Significant data requirements − Deep Neural Network models demand a substantial volume of data encompassing various traffic conditions for effective training and convergence. Moreover, the data must be extensive and diverse enough to ensure that the model's generalization capability remains uncompromised[1].

- Extended training time − The complex architecture of Deep Neural Networks, with numerous layers and numerous hyperparameters, contributes to longer training times compared to traditional statistical and simpler Machine Learning models. Additionally, updating and retraining the models' parameters when new data becomes available is a time-consuming and resource-intensive process (e.g.,[35]).

- Hyperparameter selection challenges − Determining the appropriate number of hidden layers and neurons in each hidden layer in a Neural Network often relies on experience or a trial-and-error process. Having too many neurons per hidden layer leads to an overfitting-prone network and prolonged computational times. Conversely, using too few neurons can compromise prediction accuracy, especially when dealing with a substantial volume of input data. The absence of a definitive solution for determining the optimal architecture remains a persistent issue.

These factors explain why research on DL methodologies remains an extremely active field and motivates studies like the current one. Striking a balance between model complexity, computational resources, and time is crucial when deploying Neural Networks in real-world conditions, as these factors can influence the model's accuracy. The aforementioned limitations apply not only to Neural Networks in general but also hold significance in the context of traffic data. Collecting adequate data from the entire road network over an extended period can be challenging. Moreover, the lack of interpretability can restrict the practical applicability of prediction models, as previously noted.

2.2 Quantum computing in transportation

Quantum computing has the potential to revolutionize transportation by solving computational challenges intractable for classical computers. Over the past few years, quantum computing has garnered significant attention for its potential applications in transportation optimization. Several studies have explored this promising technology for various transportation problems. For instance, the authors in[41] examined the future potential of quantum computing in ITSs, highlighting its ability to tackle complex computational problems. Bentley et al.[42] demonstrated the application of quantum computing for optimizing transport

routes, showcasing improvements in computational efficiency and solution quality. Similarly, the study in[43] applied quantum computing to transport network design problems, illustrating its advantages in handling large-scale network optimization tasks. The study by[44] focused on using quantum computing to solve scenario-based, stochastic, time-dependent, shortest-path routing problems, further proving its efficacy in dynamic and uncertain environments. Additionally, Yarkoni et al. in[45], introduced the "Quantum Shuttle" system, which utilized quantum computing for real-time traffic navigation during large events, marking a significant milestone as the first commercial application of quantum computing in traffic management.

2.3 Quantum Machine Learning

One of the most plausible candidates for exploiting the practical advantages of quantum computing in the NISQ era is QML[46]. QML offers a wide range of applications, such as utilizing data-driven approaches to discover quantum algorithms, optimizing quantum experiments, processing classical or quantum information using Quantum Neural Networks (QNNs), and even developing quantum-inspired classical Machine Learning protocols[47]. Various QML techniques, including QNNs with parameterized quantum circuits and measurements, hybrid quantum-classical schemes, and quantum heuristic algorithms, have been proposed to tackle these tasks[48–51].

In our previous works (see[52] and[53]), we provided a comprehensive discussion on the role of quantum layers in hybrid neural networks and their effect on the learning process. We addressed how QNNs can improve a task's performance by utilizing the inherent advantages of quantum mechanics, such as the ability to process information in more complex ways than classical networks. By increasing the complexity of these quantum layers, the networks could better capture and model intricate patterns in the data, potentially leading to enhanced predictive accuracy.

However, despite the rapid progress in the field, numerous open and challenging tasks remain to be addressed. These include efficient encoding data schemes for quantum processing, improving quantum models, refining training methodologies, enhancing generalization capabilities, mitigating the impact of quantum noise, and more. The present research aims to tackle several ongoing research issues in QML. Drawing on the practical advantages of quantum computing during the NISQ era, we seek to explore innovative solutions for data encoding, quantum model optimization, robust training techniques, and other related challenges. By addressing these issues, we seek to advance the capabilities of QML and unlock its full potential for real-world applications in the NISQ era.

3 Methodology

3.1 Data Description

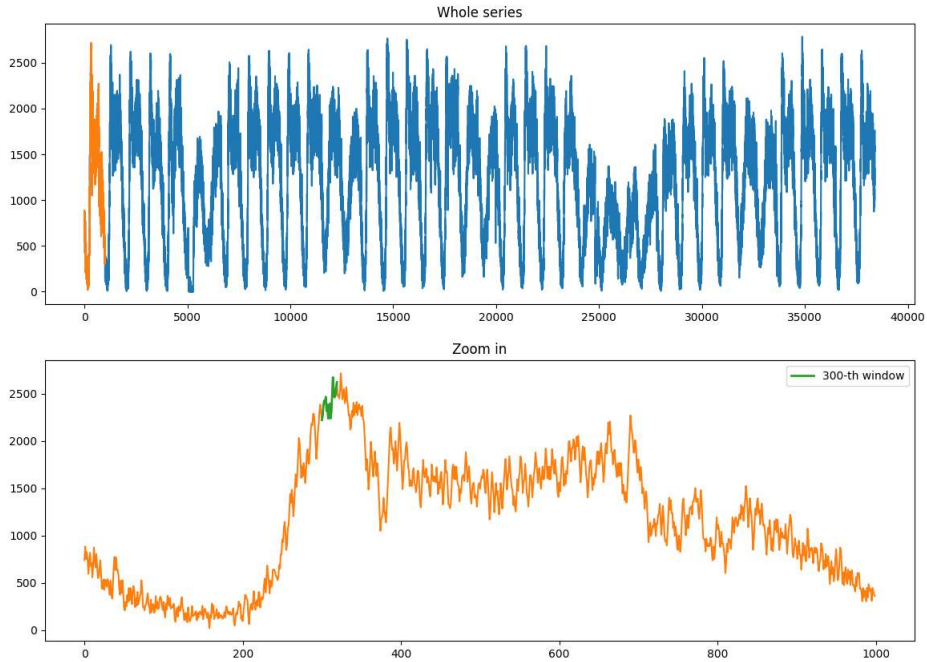

Figure 2. Full extent (*top*) and detailed view (*bottom*) of the time series used in the study. The *x*-axis displays the datapoint index, while the *y*-axis shows the traffic volume in units of vehicles/hour.

Our traffic forecasting task was based on traffic flow data representing the number of vehicles passing through a loop detector installed on Syggrou Avenue, situated in the heart of Athens, Greece. This road, which leads toward the city center, is one of the busiest in the metropolis, with three lanes accommodating over 2,000 vehicles per hour during peak times. This location was selected for this study specifically due to its high traffic volume, which makes it an ideal subject for analysis. The exact position of the loop detector is shown in Figure 1.

The data were sourced from a database jointly developed by the Greek Government and the Region of Attica, which compiles information from nearly 400 loop detectors across the region. This dataset was provided for academic research and relevant studies. For this study, we analyzed 40 days of traffic flow data collected during March and April 2023, with a time resolution of 1.5 minutes, corresponding to 40 measurements per hour. Traffic flow (or traffic volume) represents the number of vehicles passing through a specific point on the road network within a given time frame, typically expressed in vehicles per hour. This metric varies between approximately 2,000 vehicles/hour for large highways and arterials, to as low as 500 vehicles/hour for urban road sections, and even less during congested periods.

The complete time series is illustrated in Figure 2, where the top panel presents the full extent of the data. The x-axis represents a timestamp index, while the y-axis indicates vehicle flow in units of vehicles per hour. A closer view on a specific time series segment is shown in the bottom panel of the same figure, with the green section highlighting the size of a "windowed" unit used during the training phase of the neural network model (see Section 3.3).

3.2 Model Architectures

The main objective of this study is to compare classical against hybrid quantum-classic Neural Network (NN) approaches in a traffic forecasting task. The underlying goal is to assess the capacity and efficiency of quantum layers in NNs and examine whether such quantum configurations offer tangible advantages over their classical counterparts.

Quantum layers resemble classic ones in their high-level usage but fundamentally differ in their operational details. At the core of the quantum layer lies a Quantum Variational Circuit (QVC), which is central to the hybrid quantum-classical NN architecture. The QVC is designed to leverage the principles of quantum mechanics and is usually composed by the chaining of three key elements (e.g.,[54,55]):

1. *Data Embedding* — The data embedding stage is responsible for encoding the classical data into quantum states. Regardless of whether the classical data serves as the network input or comes from the preceding classical layer (referred to as the "feeding layer"), the embedding stage encodes this information into qubits. One way to do this is via angle rotation encoding, a technique where classical values are mapped to the rotational angles of qubits. These rotations prepare the quantum state that subsequent quantum operations will process. The role of this encoding step is crucial, as it directly impacts the ability of the quantum circuit to capture the nuances of the data.

2. *Entangling stage* — Following the angle embedding, the qubits undergo a series of operations in the entangling layer. As an example, in our models (see Sections 3.2.1 and 3.2.2), we will use rotational gates and Controlled-NOT (CNOT), which are building blocks for creating quantum entanglement. The CNOT gates generate entanglement between pairs of qubits, allowing the quantum layer to capture complex interdependencies within the data. The rotational gates, which have trainable parameters, adjust the quantum states further, based on the encoded data. The entangling process is a key aspect of quantum computing that enables the model to explore a much larger solution space than classical methods could, potentially leading to better generalization and more accurate predictions.

3. *Measurement Stage* — The final component of the QVC is the measurement stage, where the quantum states are measured to extract classical information from the quantum layer. Measurement collapses the quantum states into classical bits, which are then passed to the next layer in the NN. This step bridges the quantum and classical parts of the hybrid model, allowing the information processed by the quantum layer to inform the subsequent operations in the classical layers. The measurement quality directly affects the quality of the information the classical network receives, making it a critical part of the quantum-classical interface.

These operational peculiarities of the quantum layers suggest that conducting a *classical vs. hybrid* model comparison is a challenging task. In practice, this difficulty boils down to the issue that the classical equivalent to a qubit – the fundamental computational unit in quantum architectures – is not easily defined, if it can be defined at all. In fact, the two computational techniques are intrinsically different in both the way they store *and* process the information. Therefore, when performing such comparisons, one has to decide on which concept the classic–hybrid equivalence shall be based on. In this study, we opted to contrast the capabilities of classical and quantum networks whose complexities have been deemed comparable under the principles of A) size of encoded information, and B) number of recursive iterations. Namely, to explore these two scenarios, we designed two distinct experiments in which we replaced a layer of completely classic NNs with their quantum equivalents. In the first case, we focus on NNs based on fully connected layers, while in the second, we explore recursive NNs.

3.2.1 Scenario A: Layer replacement based on equal amount of encoded information

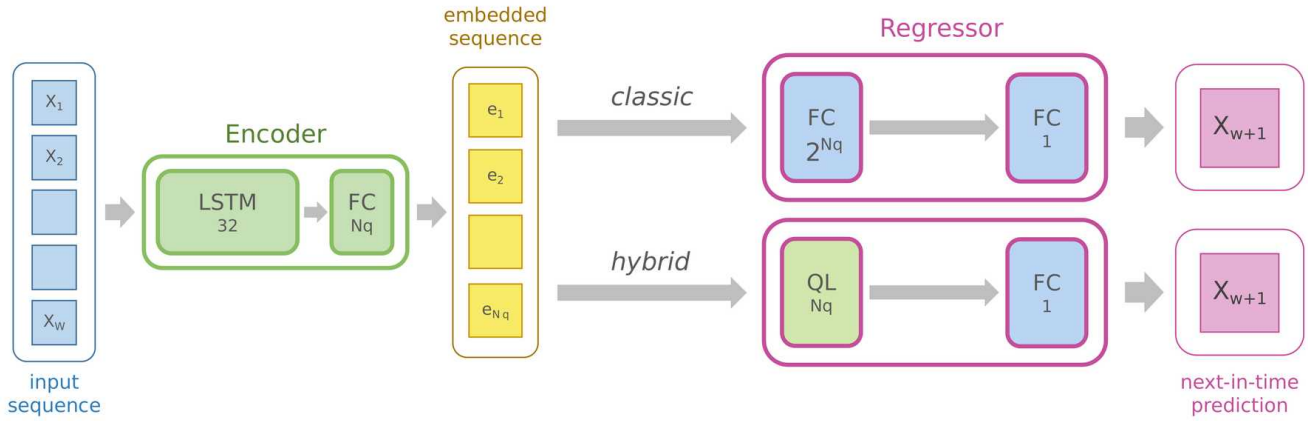

Figure 3. Schematic representation of the fully connected architecture used in the scenario described in Section 3.2.1. The left side of the image shows the encoding provided by the autoencoder trained as depicted in Figure 4. The right side represents the part of the NN that acts as a regressor. In Section 3.2.1, we compare two approaches: a fully *classic* one (*top right*), and a *hybrid* one (*bottom right*): the difference between the two lies in the first fully connected layer. In either case, the output of the NN is a single value representing the prediction at the timestep immediately following the input sequence.

In this scenario, we consider an NN in which the burden of handling the regression task falls primarily on the Fully Connected (FC) layers, which serve as the core computational components responsible for processing and refining the input data. For a detailed overview of this scenario's architectures, refer to Figure 3, which illustrates the structure and interplay of these FC layers within the broader network.

In this context, we aim to explore the effects of substituting a single, conventional, feed-forward FC layer of an NN *with* a quantum layer designed to withhold an equivalent amount of information. The underlying premise is that a quantum system, through the use of qubits, has the potential to exploit higher-dimensional embedding spaces. Specifically, N_q qubits create an embedding space of dimensionality 2^{N_q}, allowing us to theoretically replace a classical layer comprising N_q neurons with a quantum layer consisting of approximately $\sqrt{N_q}$ qubits. However, for the practical implementation, we reverse this approach (without loss of generality): we begin by constructing a quantum layer with N_q qubits and subsequently compare its performance against a classical layer containing 2^{N_q} neurons.

Exploring the performance of FC layers in a time series example is challenging because FC layers fall short when it comes to handling temporal dependencies. This makes them less suitable for tasks where the sequence and timing of data play a crucial role, as they do not inherently account for the dynamic patterns and interactions that unfold over time. To fight this limitation, we can introduce a form of preprocessing that precedes the FC layer and is capable of capturing the temporal dependencies. Specifically, we adopted an autoencoder architecture based on LSTM cells. After training, the encoder is able to process input sequences, reduce their dimensionality, and, crucially, account for time-dependent relationships. We stress that, while dimensionality reduction is not strictly necessary in this stage – its main objective being to capture the temporal properties – it becomes a beneficial side effect, as it enhances computational efficiency by reducing the overall complexity of the input, thereby speeding up subsequent calculations.

The autoencoder component is shown in Figure 4, while the complete pipeline architecture is displayed in Figure 3, and works as follows. Every "input sequence" ($x_1, x_2..., x_w$), where w is the window size (in our case, 20 timesteps), is first parsed through the pre-trained encoder to produce a compressed "embedded sequence" ($e_1, e_2..., e_{N_q}$) of size N_q. The embedded sequence is then passed to the regressor, which, as described above, can be the classical regressor, in which the first layer is an FC layer composed of 2^N_q neurons, or the hybrid regressor, in which the first layer is a quantum layer composed of N_q qubits. The final output is, either way, a single classic neuron with a linear activation function, as the NN is designed to provide for the next-in-time prediction.

3.2.2 Scenario B: Layer replacement based on equal amount of recursions

In this scenario, we attempt to go beyond fully connected layer architectures and use recursive layers for the regression task. Specifically, we design the regressor, which contains an LSTM cell, and compare it against a regressor in which this element has been replaced by a quantum layer characterized by multiple data *re-uploading*.

Data re-upload[27] is a technique initially designed to enhance the learning capacity of the model without increasing circuit

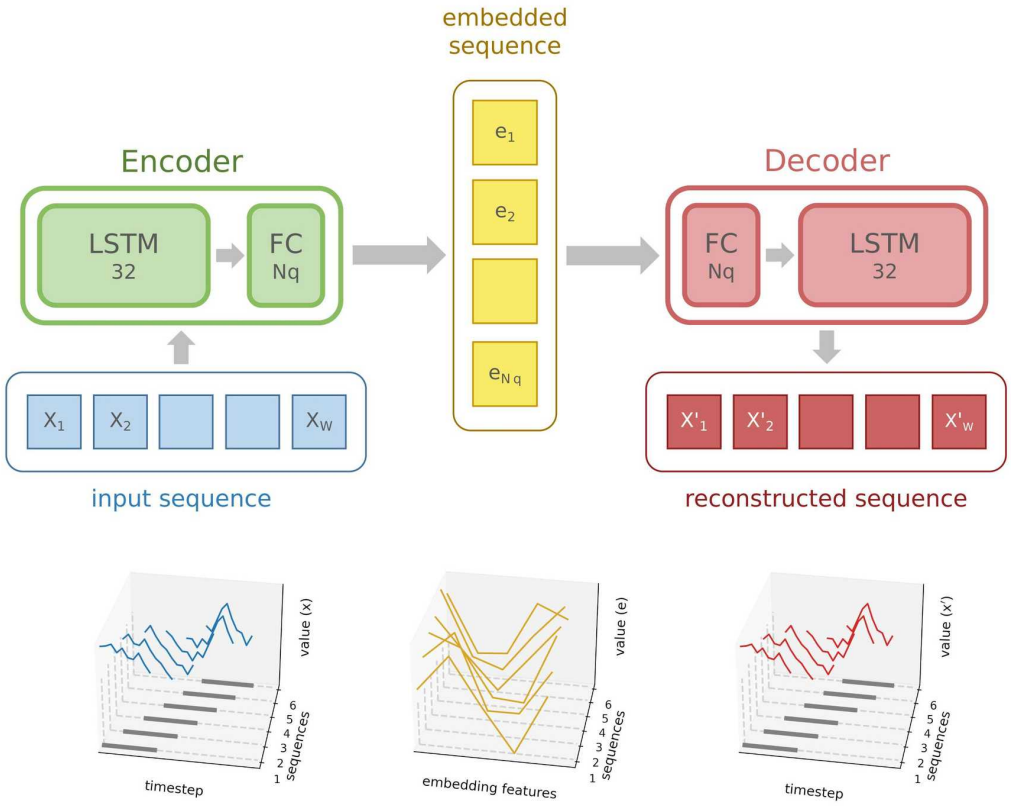

Figure 4. The architecture of the autoencoder is used as a preprocessing step for the NNs in Figures 3 and 6. The encoder is constituted by an LSTM cell composed of 32 units and an FC layer, which shrinks the embedding space to the desired N_q features; the same structure is mirrored in the decoder. The autoencoder is trained in a standard way, i.e., by matching the input sequence with the reconstructed one via minimization of the mean squared error.

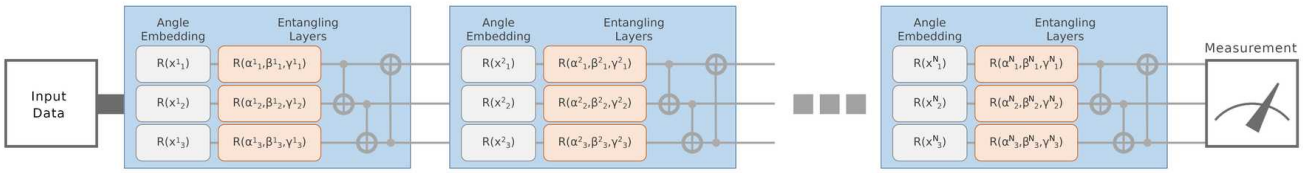

Figure 5. Example of an unfolded data re-upload scheme with N re-upload blocks (blue boxes) with the same components as the ones adopted in this work. In this specific depiction, the classic input data have dimensionality 3, and the circuit is composed of 3 qubits. In each block, the grey boxes represent the rotations used to embed the classical data, and two entangling layers follow them. The first such layer is composed of rotational gates (orange boxes), each characterized by three tunable parameters (α, β, and γ), while the second layer is composed of CNOT gates. At the output of the circuit, a measurement converts the signal back to classical data (e.g., in our work, we applied a Pauli-Z measurement to observe the state of the qubits along the Z-axis in the computational basis). In this representation, variables and parameters are indexed as $\langle \cdot \rangle_q^n$, where n is the block index, and q the qubit index. In general, multiple entangling layers may be chained inside a single block to increase its complexity.

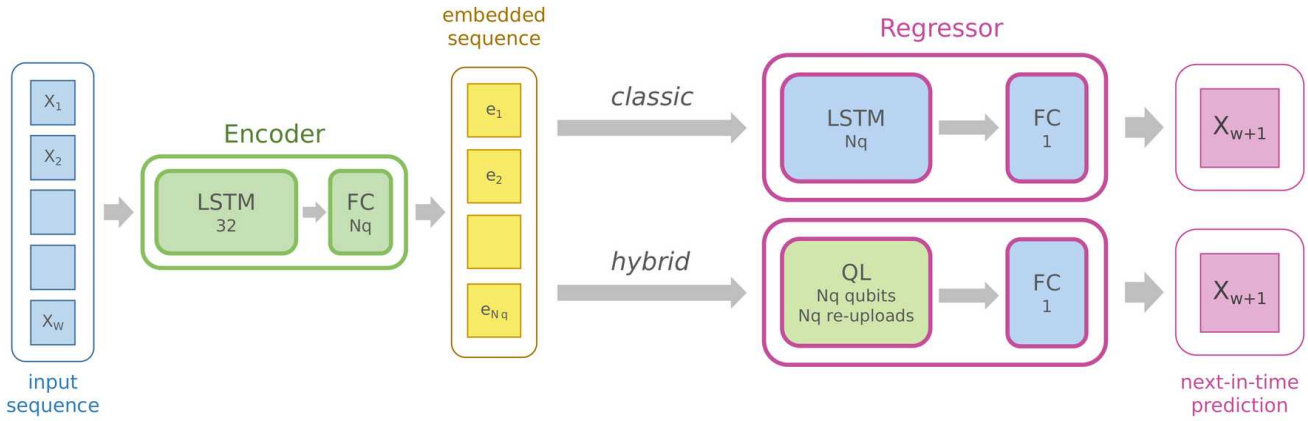

Figure 6. Schematic representation of the recursive architecture used in the scenario described in Section 3.2.2. The left side of the image shows the encoding provided by the autoencoder trained as depicted in Figure 4. The right side represents the part of the NN which acts as a regressor. In Section 3.2.2, we compare two approaches: a fully *classic* one (*top right*), and a *hybrid* one (*bottom right*): the difference between the two lies in the first layer. In the *classic* approach, the first layer is an LSTM cell composed of N_q units, while in the *hybrid* case, it is a quantum layer with N_q re-uploads and N_q qubits. In either case, the output of the NN is a single value representing the prediction at the timestep immediately following the input sequence.

depth or qubit count. It works by encoding the same classical data into the quantum circuit multiple times, as depicted in the example of Figure 5 and described by the following pseudo-algorithm:

Algorithm 1 Data Re-upload Pseudo-Algorithm

Initialize quantum state $|\psi\rangle$
for each re-upload r **do**
  1. Encode classical data x using parameterized gates
    $R_x(x,\theta_r), R_y(x,\theta_r)$
  2. Apply quantum operations
    (e.g., rotations, entangling gates)
end for
Measure final quantum state

Intuitively, since each re-upload uses different parameters for the tunable gates (θ_r) at each iteration, data re-uploading allows to explore more complex patterns. Not surprisingly, since its first appearance, this technique acquired immediate success due to its potential to improve performance, and it has been applied in diverse classification and regression tasks (e.g.,[56] and references therein).

However, data re-upload has been exploited so far primarily for its potential to explain complex patterns; in this work, we intend to shift the attention to another property of this methodology. In fact, note that the same classical data are iteratively layered onto an already evolved quantum state (modified by the "quantum operations" in between re-uploads). Therefore, data re-uploading does not *erase* the previous quantum state, but rather *adds* to it. In this sense, while the quantum system retains the previously acquired quantum information, the re-uploading allows further manipulation of the state. This technique, hence, works in a similar fashion to recursive memory cells in classic NNs, such as LSTMs.

The complete pipeline architecture used in this experiment is displayed in Figure 6. As in Scenario A, the input sequence is first parsed through the pre-trained encoder to produce a compressed embedded sequence, which is, in turn, passed to the regressor. In the classic regressor, the first layer is an LSTM layer composed of N_q units; notice that, since the embedded sequence is also N_q timesteps long, this LSTM will recurse N_q times. Similarly, the first layer of the hybrid regressor is a quantum layer acting on N_q qubits and performing N_q re-uploads (re-uploading "blocks")[1]. Notice that, in our experiments, each re-uploading block consists of an angle embedding layer followed by a single entangling layer, although, in principle, one could add multiple entangling layers in each block.

[1]This limitation, i.e., that of picking N_q both for the number of qubits *and* the number of re-uploads, is dictated by the way the data re-uploading in hybrid networks is implemented in Pennylane – see Section 3.2.3.

We emphasize that, by configuring the networks in this manner, we effectively propose an innovative solution to the challenge of comparing quantum and classical recursive networks. In fact, the LSTM and quantum elements that we designed are equivalent in the sense that both take as input N_q-long sequences, produce N_q-long sequences as outputs, and recurse N_q times. The final output is either way a single classic neuron with a linear activation function, as the NN is designed to provide for the next-in-time prediction.

3.2.3 Further insights on the quantum layers

Simulating quantum layers on classical hardware is a computationally intensive task, especially as the number of qubits increases. Simulating more than 10 qubits can quickly become unfeasible on standard personal computers, making it necessary to strike a balance between the model's complexity and the computational resources required. For this reason, we kept the quantum layers relatively small in terms of qubits ($N_q \leq 14$), acknowledging that this choice might come at the expense of the model's potential performance. However, as demonstrated in our results (Section 4.2), our model architecture achieved extremely high-performance scores, primarily due to the abundance of training data available.

The quantum layers were implemented using Pennylane[2], a Python-based tool specifically designed for quantum machine learning and optimizing hybrid quantum-classical computations[52,57]. Pennylane provides the necessary infrastructure to simulate quantum circuits on classical computers, making it possible to explore the potential of quantum computing in practical applications, even without access to physical quantum hardware.

Note that, in the remainder of this manuscript, the classic–hybrid architecture pairs will be identified by the number of qubits in the *hybrid* neural network (e.g., 'Q2' indicates 2 qubits), with the number of neurons in the corresponding layer of the classic NN being derived as described in Sections 3.2.1 and 3.2.2.

3.3 Training

The NNs described earlier are designed to predict the value of the series at a specific timestep $t+1$ based on the values from the previous w timesteps $(x_{t-w}, x_{t-(w-1)}, \ldots, x_t)$. To achieve this, the dataset is divided into "windows", which are small sequences of w data points, with each window serving as a predictive variable array. The next data point in time, immediately following each window, is the target variable the model attempts to predict.

The training process involves feeding these windows and their corresponding target values into the network in mini-batches. This approach helps stabilize and accelerate the learning process, especially when dealing with large datasets. An Adam optimizer with a learning rate of 0.0005 has been adopted to minimize the Mean Squared Error (MSE) – the loss function – helping the network learn the mapping between input sequences and their corresponding targets. The prediction stage follows the same rationale, with the $t+1$ test data point being predicted given the previous w values.

3.4 Performance Evaluation

Regarding the testing, we strived to obtain an unbiased estimate of the uncertainty about the results in order to quantify the difference between the NNs. In time series forecasting modeling, it is common to segregate a test set that comes temporally after the training set. This approach is most diffused because of the implicit assumption that time series data points are timely correlated and potentially causally correlated. It follows that the value of the series at a given time t is strongly associated with that at time $t-1$.

However, while it is not controversial to segregate a single hold-out test (at the end of the series) for an individual assessment, placing an uncertainty on this quantity is not straightforward. In fact, this requires performing multiple train/test splits – hence raising the question: where should the different test sets be selected along the time series? In tabular data multiple testing is readily solved by using cross-validation (CV), in which the data ensemble is split in multiple folds, and one of the folds plays the role of test set (while the rest play the training), until all permutations are covered. However, because of the aforementioned time-dependency issues, the same approach is not deemed applicable right away to time series, and, in general, there seems to be an open debate on what the best solution might be (see, e.g.,[58] and references therein).

In this study, we opted for a 5-fold gap-cross-validation technique[59] via the GapKFold[3] Python implementation[60]. This protocol is similar to the CV, with the significant difference that there are "gaps" (i.e., discarded data) on both sides of the test fold. In other words, at each CV permutation, part of the data contiguously preceding and following the test fold are ignored. Figure 7 shows a graphical representation of our own variant of this protocol, in which, at each folding, we also insert a validation set[4]. In summary, the core idea of this methodology is that discarding temporally adjoined data guarantees, to some extent, a break in the causal dependence between training and test, providing independent sets. Notice that this technique uses – as training data – folds that may be located temporally after the test set.

[2] https://pennylane.ai
[3] https://github.com/WenjieZ/TSCV
[4] The validation data are solely used to monitor the NN training.

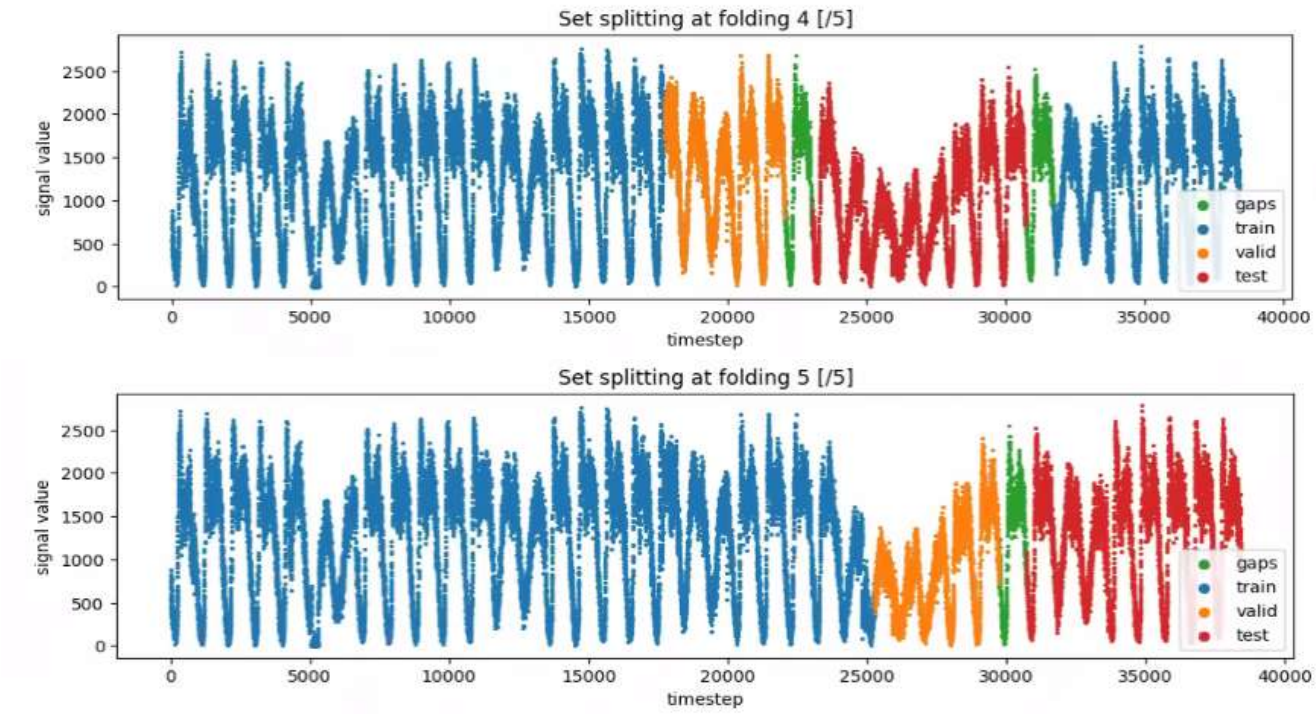

Figure 7. Two iterations of the gap-cross-validation protocol were employed in this work to assess the performance uncertainties. The train, validation, and test folds are represented in blue, orange, and red, respectively. The green slices show the gaps excluded from the analysis at each folding. The validation set is forced always to precede the test set, while the training set can be either earlier or later than the test set. The test set is always a unique block, but the other sets can wrap around. The gap is not necessary if the test fold appears at the edge of the series (e.g., bottom panel).

The sequences (windows) used for the actual training (see Section 3.3 and Figure 2, bottom panel) are generated after the train/validation/test splicing to guarantee that there is no data leakage across the sets.

4 Results

4.1 Training convergence

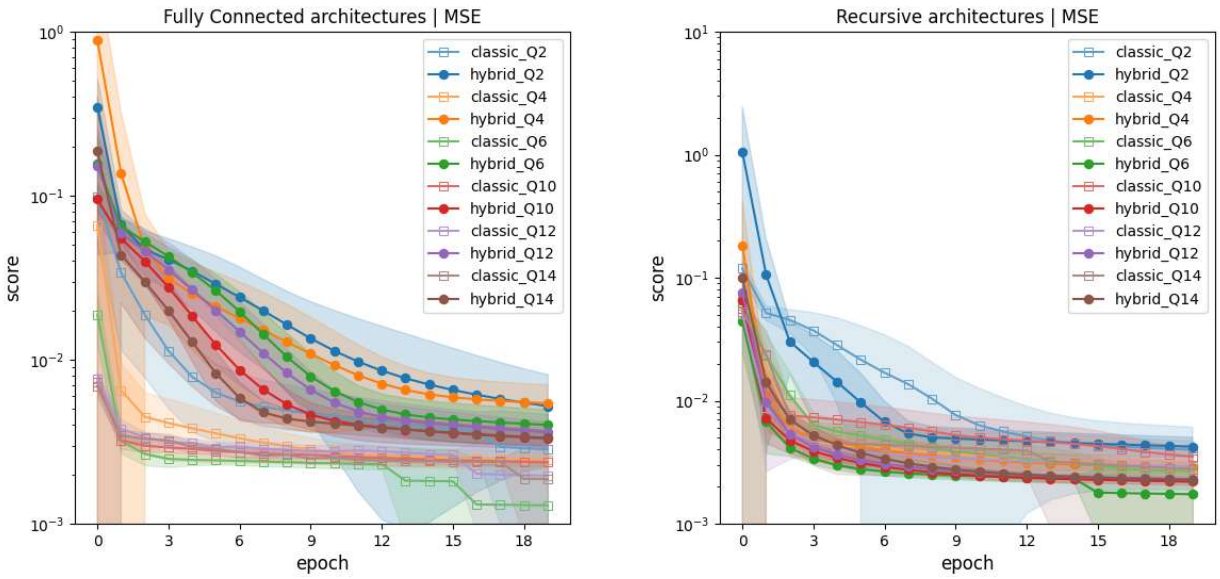

Figure 8. Evolution of the models' loss (MSE) as measured on the training sets for the fully connected architectures (Scenario A; *left*) and the recursive architectures (Scenario B; *right*). The curves represent the mean value at each epoch, averaged over the 5-fold cross-validation. The shaded areas, corresponding to the color of each curve, indicate the 1-σ standard deviation around the mean. Empty circles represent *classic* architectures, while filled circles of the same color refer to the corresponding *hybrid* ones. The scores refer to the normalized data values.

One of the key challenges of this study is to compare the computational speed of classic NNs as opposed to hybrid ones, i.e., examine which one converges faster. Due to the significant difference in computational requirements – given that quantum layers are being simulated on classical hardware – a direct comparison of wall clock times would be unfair. Instead, our study compares the speed of convergence in terms of training epochs, providing a more meaningful metric in this context. In Figure 8 we show the evolution of the training loss (MSE) across the epochs, for each model considered in this work.

Two main considerations emerge from this figure. In the first place, we notice that both fully connected and recursive architectures manage to converge within the 20 epochs we experimented on, and moreover converge to very similar scores. This relatively rapid convergence is due, more than to a complex network architecture, to the extremely large wealth of data available for our series. We had about 40 000 data points, which we split into windows of 20 data points each, ultimately resulting in approximately 800 batches (composed of 32 windows each) per epoch. Regarding the similar score values, consider that this dataset exhibits an extremely regular pattern, and that the task is a next-in-time forecasting challenge, where the goal was to predict a single future data point per window. These factors greatly simplified the forecasting problem, thereby explaining the consistently high performance across all NNs. However, note that while the curves of all recursive models (except for Q2) flatten at around epoch 5, the curves for the fully connected architectures present a larger spread.

This is connected with the second observation, i.e., that the hybrid models in fully connected architectures converge way slower than their corresponding classical ones. This trend is reversed in the recursive architectures where, although less significantly, the hybrid models converge faster than their counterparts, especially for smaller numbers of qubits.

4.2 Performance Scores

As assessment metrics, we considered a collection of commonly adopted regression metrics, namely MSE (also used as a loss function, see Section 3.3), Mean Absolute Error (MAE), and R^2 (coefficient of determination).

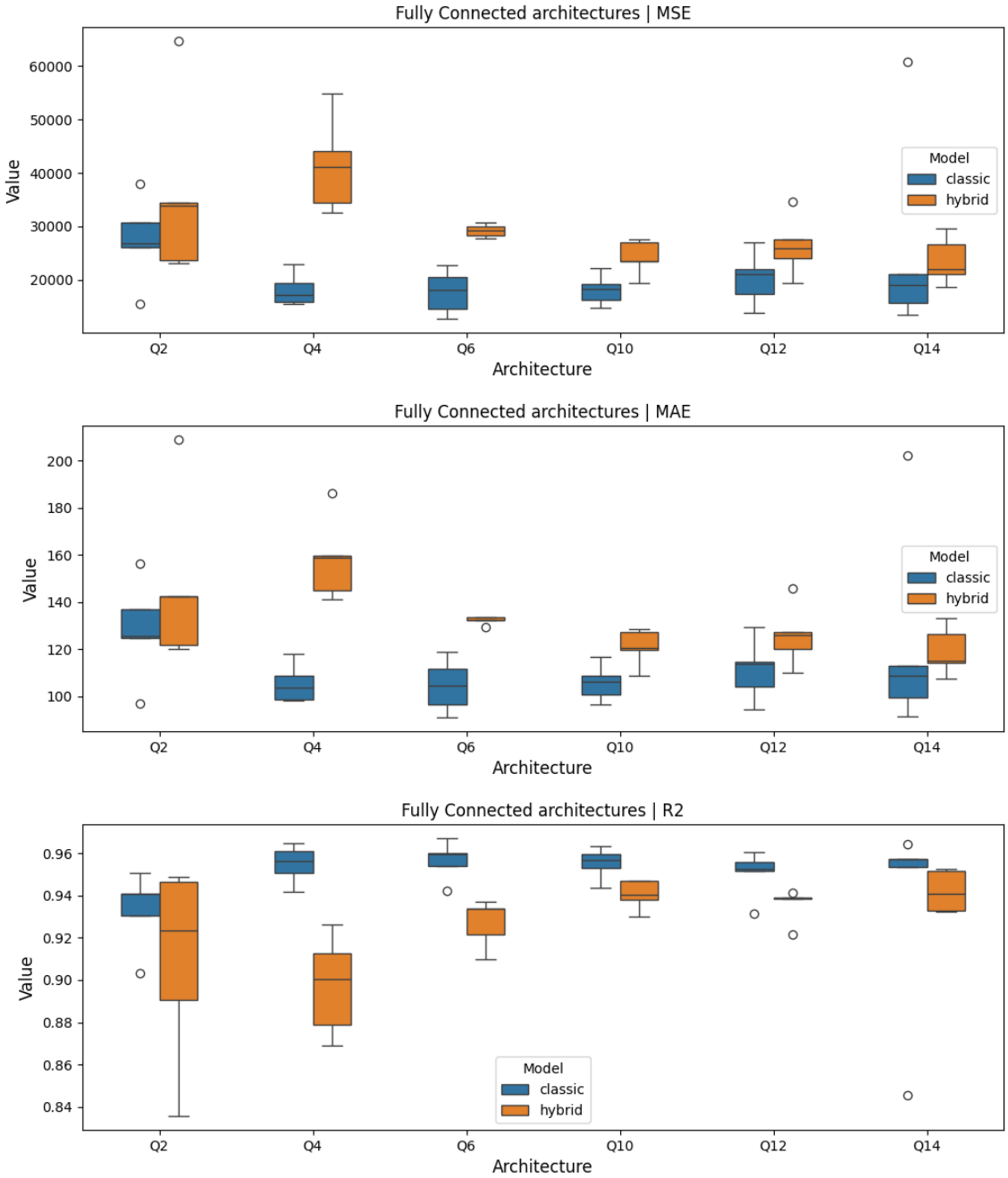

Figure 9. Performance scores of all fully connected models explored in this study, as evaluated on the 5 test (hold-out) folds of the CV. From *top* to *bottom*: Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2. The boxes represent the interquartile range (IQR), which is the range between the 25*th* and 75*th* percentiles of the data. The horizontal line inside the box indicates the median value. The whiskers extend from the edges of the box to a distance of 1.5 times the IQR, with outliers represented by empty circles. Corresponding *classic* (blue)–*hybrid* (orange) models are reported next to each other and indexed by the number of qubits (Q$\langle N_q \rangle$).

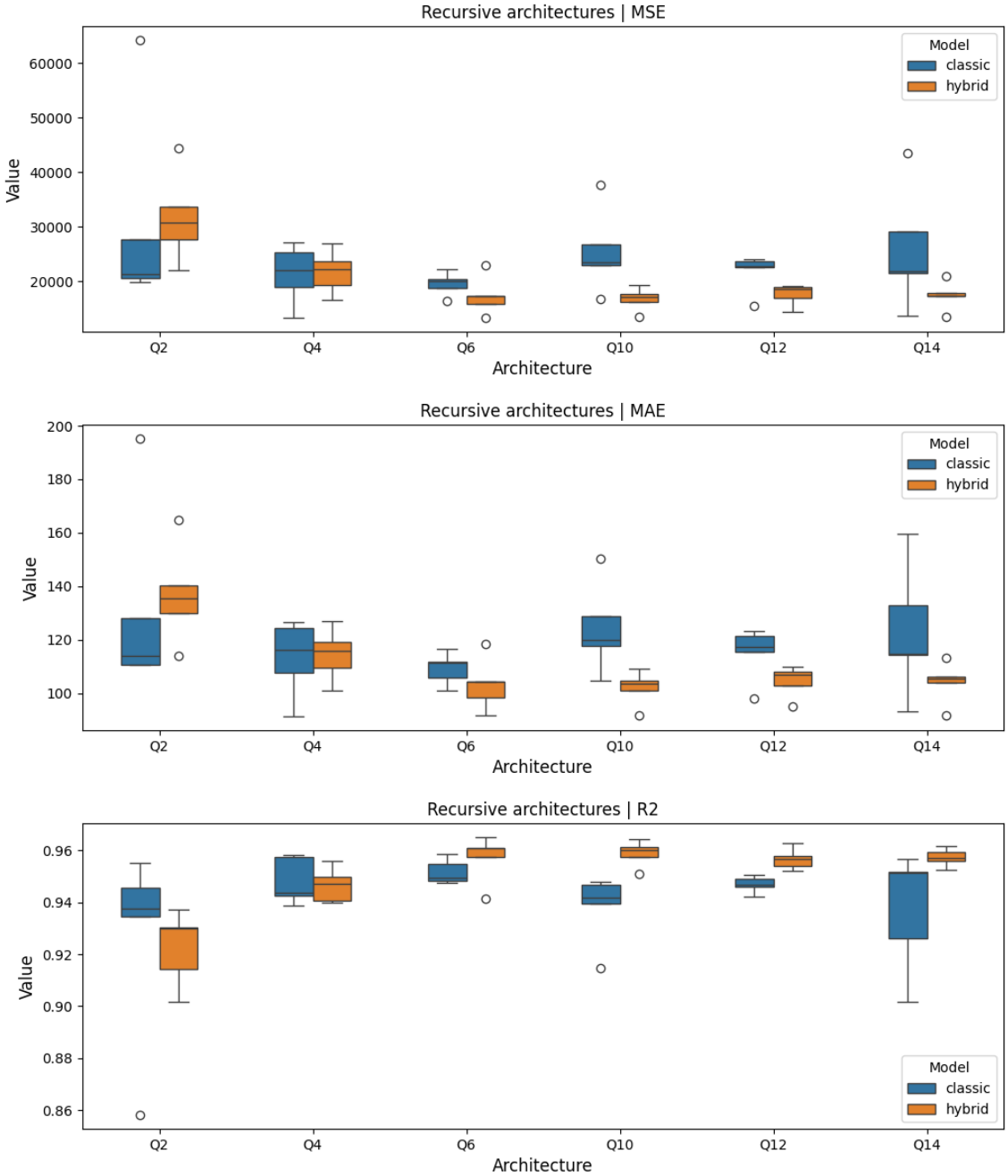

Figure 10. Performance scores of all recursive models explored in this study, as evaluated on the 5 test (hold-out) folds of the CV. Same as for Figure 9.

Figure 9 shows the measurements relative to the different metrics, estimated over the test folds of the CV loop, for all the fully connected architectures (Scenario A). From the figure it emerges that, independently of the metric, the discrepancy between classical and hybrid architectures becomes more significant as the number of qubits(/neurons) decreases (except for the case of Q2, which is arguably dominated by outliers, as demonstrated by the sudden drop of performance even for the classic model). In particular, we observe that the classic/hybrid scores start to be compatible, i.e. within their respective uncertainties, from 10 qubits onward.

Figure 10 shows the same measurements, this time estimated for the recursive architectures (Scenario B). In this case, we observe that the hybrid architectures outperform the classical counterparts for 6 qubits or more; moreover, they present lower dispersions, indicating that they are more consistent with respect to the variation of the train/test sets.

4.3 Consistency Check

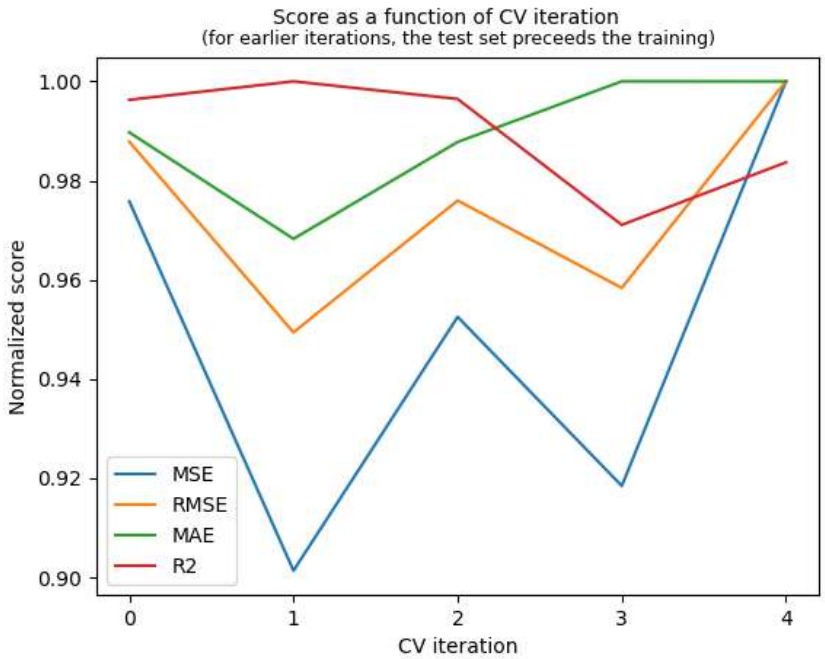

Figure 11. Assessment of a collection of metrics on the test sets belonging to different folding of the cross-validation protocol illustrated in section Section 3.4

To rule out any potential bias introduced by our CV assessment protocol, which involved test sets preceding the training sets (as discussed in Section 3.4), we conducted an additional test. We evaluated the performance scores as a function of the test set's position relative to the training set, as shown in Figure 11. Note that, despite this specific plot refers to the hybrid model Q6, it is representative of the results we obtained for all the other models. The scores have been normalized to 1 for visualization purposes; our interest lies primarily in the trend rather than the absolute values.

Intuitively, Figure 11 explores whether 'peeking into the future' – i.e., placing the test set before the training set – results in better performances, as would be anticipated if a bias was indeed present. In such a case we would expect a decline in performance from left to right on the plot, since the GapKFold protocol positions the test set before the training set in earlier iterations (and gradually shifts it to the end in later iterations). The figure indicates instead that the scores are largely independent of the iteration loop, oscillating around an average value. This allows us to rule out any significant flaws in our assessment protocol.

5 Discussion

The most immediate result of our investigation – as it readily emerges by the comparison between the left and right panels of Figure 8, and the comparison between Figures 9 and 10 – is that quantum-based NNs do not appear convenient in fully connected architectures, but do offer advantages in recursive architectures (i.e., when data re-upload is involved). Granted, this conclusion is valid under the assumptions of architecture equivalence defined in Sections 3.2.1 and 3.2.2.

Surprisingly, despite data re-uploading having operational similarities with RNNs (see Section 3.2.2) − an industry-standard in time series analysis (see, e.g.,[61] for quantum-LSTMs) − there is little literature exploring its application in this context. To the best of our knowledge,[62] is the only work applying data re-upload on a pragmatic time series example. In this work, the authors study a hybrid encoder-decoder and show that it is superior, in terms of performance, to its classical 'equivalent'. In[62], the classical equivalent is a Seq2Seq architecture[63], while the hybrid model is composed by the same Seq2Seq extended by a quantum circuit including data re-upload.

In our Scenario B approach, we expand on their methodology by *a*) basing the comparison on recursive loops rather than network shape, and *b*) keeping the classical component (in our case, the LSTM autoencoder) as small as possible, using only the encoder and solely for data compression. These improvements allowed us to focus more clearly on the individual contribution of the data re-upload layer. Additionally, we explored a range of data re-upload iterations (from 2 to 14), emphasizing the novel exploitation of this technique: its usage as a memory cell − rather than merely as a method to enhance a layer's expressivity, as it has so far been employed in regression/classification tasks.

Secondarily, we observe that – regardless of the comparison scenario – the relative performance of the classic NNs with respect to their hybrid counterparts rapidly declines as network size increases. Specifically, as the number of qubits increases, the hybrid NNs in fully connected scenarios catch up with the classical ones (Figure 9), whereas in recursive architectures, the hybrid NNs progressively diverge towards better performances (Figure 10). Moreover, we generally observe that this relative performance is non-linear with respect to the number of qubits, evolving more rapidly for lower qubit counts.

Adding to these considerations, we draw again attention to how the training curves of Figure 8 all flatten out before the limiting 20 epochs are reached. This provides an indication that the aforementioned performance differences are not due to a lack of convergence but are *indeed* related to the models' generalization prowess. However, focusing on the fully connected architectures (Scenario A; *left* panel) we discover that, while the classical models basically converge within 3-4 epochs, the hybrid models take progressively longer for decreasing qubits counts. This further suggests that the issue relies on an inferior modeling power of hybrid fully connected architectures. In contrast, the hybrid architectures exploiting data re-upload (Scenario B; *right* panel) seem to provide the ideal trade, as the models converge within 5 epochs while reaching comparable or even better performances than the fully connected hybrid NNs.

It is worth noting that these figures are based on simulations of quantum neural networks on conventional hardware, where data re-upload is significantly more computationally expensive. In our experiments, training a quantum model with N_q qubits took approximately two times longer for each data re-uploading block involved (e.g.,[64]). Nevertheless, our results offer valuable insights into the resource demands and optimization strategies necessary when implementing quantum algorithms on actual quantum computers.

6 Conclusion and Future Directions

In this paper, we present one of the pioneering efforts to explore and quantify the impact of quantum computing and quantum machine learning on traffic forecasting. Our results show that a relatively simple quantum-hybrid model can achieve performance levels comparable to an optimized classical LSTM-based network, though with higher computational complexity. Most importantly, our findings arguably provide one of the first pieces of evidence suggesting that more sophisticated quantum models, particularly those incorporating data re-uploading, have the potential to surpass classical approaches in terms of performance in the field of time series analysis. Due to current hardware limitations, we could not fully evaluate these more advanced quantum models for a large number of qubits, but exploring their performance remains a key focus for future research.

This work lays the groundwork for the application of Quantum Machine Learning in traffic forecasting and other areas of transportation engineering. The methodologies and insights derived from this study pave the way for further exploration and optimization of quantum-based models for a range of transportation-related challenges. In these regards, we identify several important avenues to extend our studies in the future. For instance, quantum systems are inherently sensitive to errors and noise, which pose significant challenges to the scalability of quantum computers. This sensitivity impacts both qubits and quantum gates, leading to potential inaccuracies in computations. To study the impact of noise on quantum systems, researchers often use models such as the Lindblad master equation[65] or Kraus operators[66]. The Lindblad equation describes the dynamics of open quantum systems under the Born-Markov approximation, while Kraus operators provide a general framework for representing noise channels affecting density matrices. In simple terms, a noise channel is a linear map that transforms density matrices to other set of density matrices, capturing the effects of decoherence and imperfections in quantum systems. Since most of our codes have been written using the open-source package PennyLane[67], we provide general remarks on how to implement noise within this framework. Pennylane offers several methods for simulating noise in quantum circuits. These include classical parametric randomness, the built-in `default.mixed` device, and plugins for interfacing with other platforms like Cirq and Qiskit. These tools allow researchers to explore how noise affects quantum algorithms and to develop strategies for mitigating

its impact. In summary, gaining a deeper understanding of how noise models affect quantum algorithms can lead to the development of more resilient quantum computing solutions, enhancing their practical applicability across various domains.

An other aspect which offers margin for improvement regards the optimization of learning gradients in non-convex problem structures, an open challenge in quantum machine learning with no established solutions. One of the most significant hurdles is the so-called "Barren plateau"[68], which refers to the issue of vanishing gradients in the optimization landscape of quantum circuits, particularly when scaling to larger system size. Currently, the only types of QNN known to be free from this issue are quantum convolutional neural networks. In light of our current study, we would like to stress that great training results obtained through data re-uploading can be attributed to the fact that data re-uploading variational quantum circuits are, in some instances, barren plateau-free. Effectively, these circuits allow to escape from local minima by repeatedly re-introducing data into the network [69,70]. For instance, an empirical study conducted in[70] demonstrated that the magnitude and variance of the gradients remain substantial throughout training, even as the number of qubits increases. This behavior contrasts with the typical degradation seen in barren plateaus. Alternative approaches to addressing the vanishing gradient issue include characterizing the loss function landscape through Hessian computation, which can provide valuable insights[71,72]. By analyzing the eigenvalues of the Hessian, which quantify local curvature, it is possible to adapt the learning rate to achieve faster convergence during training. These techniques hold promise for mitigating optimization challenges and improving the efficiency of quantum training processes.

Looking ahead, our future research will focus on more generalizable comparisons between classical and quantum approaches, particularly in more complex scenarios involving difficult datasets with fewer observations, as well as multi-input and multi-output configurations. Additionally, we plan to assess both approaches in terms of generalizability and transferability, employing a more inclusive and stringent assessment protocol to fully understand the capabilities and limitations of Quantum Machine Learning in real-world applications.

References

1. Yin, X. *et al.* Deep learning on traffic prediction: Methods, analysis and future directions. *IEEE Transactions on Intell. Transp. Syst.* 1–15, DOI: 10.1109/TITS.2021.3054840 (2021).
2. Lana, I., Del Ser, J., Velez, M. & Vlahogianni, E. I. Road traffic forecasting: Recent advances and new challenges. *IEEE Intell. Transp. Syst. Mag.* **10**, 93–109, DOI: 10.1109/MITS.2018.2806634 (2018).
3. Cheng, Z., Pang, M. S. & Pavlou, P. A. Mitigating traffic congestion: The role of intelligent transportation systems. *Inf. Syst. Res.* **31**, 653–674 (2020).
4. Gao, Y., Zhou, C., Rong, J., Wang, Y. & Liu, S. Short-term traffic speed forecasting using a deep learning method based on multitemporal traffic flow volume. *IEEE Access* **10**, 82384–82395 (2022).
5. Vlahogianni, E. I., Karlaftis, M. G. & Golias, J. C. Short-term traffic forecasting: Where we are and where we're going. *Transp. Res. Part C: Emerg. Technol.* **43**, 3–19, DOI: 10.1016/j.trc.2014.01.005 (2014).
6. Yao, H., Tang, X., Wei, H., Zheng, G. & Li, Z. Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 5668–5675, DOI: 10.1609/aaai.v33i01.33015668 (2019).
7. Mantouka, E., Barmpounakis, E., Vlahogianni, E. & Golias, J. Smartphone sensing for understanding driving behavior: Current practice and challenges. *Int. J. Transp. Sci. Technol.* **10**, 266–282, DOI: 10.1016/j.ijtst.2020.07.001 (2021).
8. Vlahogianni, E. I. & Barmpounakis, E. N. Driving analytics using smartphones: Algorithms, comparisons and challenges. *Transp. Res. Part C: Emerg. Technol.* **79**, 196–206, DOI: 10.1016/j.trc.2017.03.014 (2017).
9. Jiang, W. & Luo, J. Graph neural network for traffic forecasting: A survey. *IEEE Transactions on Intell. Transp. Syst.* (2021).
10. Bharti, K. *et al.* Noisy intermediate-scale quantum algorithms. *Rev. Mod. Phys.* **94**, 015004 (2022).
11. Preskill, J. Quantum computing in the nisq era and beyond. *Quantum* **2**, 79 (2018).
12. Arute, F. *et al.* Quantum supremacy using a programmable superconducting processor. *Nature* **574**, 505–510 (2019).
13. Harrow, A. W. & Montanaro, A. Quantum computational supremacy. *Nature* **549**, 203–209 (2017).
14. Zhong, H.-S. *et al.* Quantum computational advantage using photons. *Science* **370**, 1460–1463 (2020).
15. Bluvstein, D. *et al.* A quantum processor based on coherent transport of entangled atom arrays. *Nature* **604**, 451–456 (2022).
16. Bravyi, S. *et al.* High-threshold and low-overhead fault-tolerant quantum memory. *Nature* **627**, 778–782 (2024).

17. Biamonte, J. *et al.* Quantum machine learning. *Nature* **549**, 195–202 (2017).

18. Cerezo, M., Verdon, G., Huang, H.-Y., Cincio, L. & Coles, P. J. Challenges and opportunities in quantum machine learning. *Nat. Comput. Sci.* **2**, 567–576 (2022).

19. Bowles, J., Ahmed, S. & Schuld, M. Better than classical? the subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059* (2024).

20. Harrow, A. W., Hassidim, A. & Lloyd, S. Quantum algorithm for linear systems of equations. *Phys. review letters* **103**, 150502 (2009).

21. Huang, H.-Y., Bharti, K. & Rebentrost, P. Near-term quantum algorithms for linear systems of equations. *arXiv preprint arXiv:1909.07344* (2019).

22. Morales, M. E. *et al.* Quantum linear system solvers: A survey of algorithms and applications. *arXiv preprint arXiv:2411.02522* (2024).

23. Rebentrost, P., Steffens, A., Marvian, I. & Lloyd, S. Quantum singular-value decomposition of nonsparse low-rank matrices. *Phys. review A* **97**, 012327 (2018).

24. Lloyd, S., Mohseni, M. & Rebentrost, P. Quantum principal component analysis. *Nat. physics* **10**, 631–633 (2014).

25. Khan, T. M. & Robles-Kelly, A. Machine learning: Quantum vs classical. *IEEE Access* **8**, 219275–219294 (2020).

26. Rivera-Ruiz, M. A., Mendez-Vazquez, A. & López-Romero, J. M. Time series forecasting with quantum machine learning architectures. In *Mexican International Conference on Artificial Intelligence*, 66–82 (Springer Nature, Cham, Switzerland, 2022).

27. Easom-Mccaldin, P., Bouridane, A., Belatreche, A. & Jiang, R. On depth, robustness and performance using the data re-uploading single-qubit classifier. *IEEE Access* **9**, 65127–65139 (2021).

28. Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E. & Latorre, J. I. Data re-uploading for a universal quantum classifier. *Quantum* **4**, 226 (2020).

29. Pérez-Salinas, A., López-Núñez, D., García-Sáez, A., Forn-Díaz, P. & Latorre, J. I. One qubit as a universal approximant. *Phys. Rev. A* **104**, 012405 (2021).

30. Zhao, Z., Chen, W., Wu, X., Chen, P. C. Y. & Liu, J. Lstm network: A deep learning approach for short-term traffic forecast. *IET Intell. Transp. Syst.* **11**, 68–75, DOI: 10.1049/iet-its.2016.0208 (2017).

31. Bapaume, T., Côme, E., Roos, J., Ameli, M. & Oukhellou, L. Image inpainting and deep learning to forecast short-term train loads. *IEEE Access* **9**, 98506–98522 (2021).

32. Wang, Y., Zhang, D., Liu, Y., Dai, B. & Lee, L. H. Enhancing transportation systems via deep learning: A survey. *Transp. Res. Part C: Emerg. Technol.* **99**, 144–163, DOI: 10.1016/j.trc.2018.12.004 (2019).

33. Fafoutellis, P., Vlahogianni, E. I. & Del Ser, J. Dilated lstm networks for short-term traffic forecasting using network-wide vehicle trajectory data. In *Proceedings of the 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, DOI: 10.1109/ITSC45102.2020.9294752 (2020).

34. Ranjan, N., Bhandari, S., Zhao, H. P., Kim, H. & Khan, P. City-wide traffic congestion prediction based on cnn, lstm and transpose cnn. *IEEE Access* **8**, 81606–81620 (2020).

35. Boukerche, A. & Wang, J. Machine learning-based traffic prediction models for intelligent transportation systems. *Comput. Networks* **181**, 107530, DOI: 10.1016/j.comnet.2020.107530 (2020).

36. Ma, X. *et al.* Learning traffic as images: A deep convolutional neural network for large-scale transportation network speed prediction. *Sensors* **17**, DOI: 10.3390/s17040818 (2017).

37. Dai, X. *et al.* Deeptrend 2.0: A light-weighted multi-scale traffic prediction model using detrending. *Transp. Res. Part C: Emerg. Technol.* **103**, 142–157, DOI: 10.1016/J.TRC.2019.03.022 (2019).

38. Leiser, N. & Yildirimoglu, M. Incorporating congestion patterns into spatio-temporal deep learning algorithms. *Transp. B* **9**, 622–640, DOI: 10.1080/21680566.2021.1922320 (2021).

39. Cui, Z., Ke, R., Pu, Z., Ma, X. & Wang, Y. Learning traffic as a graph: A gated graph wavelet recurrent neural network for network-scale traffic prediction. *Transp. Res. Part C: Emerg. Technol.* **115**, 102620, DOI: 10.1016/j.trc.2020.102620 (2020).

40. Yu, J. J. Q., Markos, C. & Zhang, S. Long-term urban traffic speed prediction with deep learning on graphs. *IEEE Transactions on Intell. Transp. Syst.* DOI: 10.1109/TITS.2021.3069234 (2021).

41. Wang, S., Pei, Z., Wang, C. & Wu, J. Shaping the future of the application of quantum computing in intelligent transportation system. *Intell. Converged Networks* **2**, 259–276 (2021).

42. Bentley, C. D., Marsh, S., Carvalho, A. R., Kilby, P. & Biercuk, M. J. Quantum computing for transport optimization. *arXiv preprint arXiv:2206.07313* (2022).

43. Dixit, V. V. & Niu, C. Quantum computing for transport network design problems. *Sci. Reports* **13**, 12267 (2023).

44. Dixit, V. V., Niu, C., Rey, D., Waller, S. T. & Levin, M. W. Quantum computing to solve scenario-based stochastic time-dependent shortest path routing. *Transp. Lett.* 1–11 (2023).

45. Yarkoni, S. *et al.* Quantum shuttle: traffic navigation with quantum computing. In *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*, 22–30 (2020).

46. Brooks, M. Beyond quantum supremacy: The hunt for useful quantum computers. *Nature* **574**, 19–22 (2019).

47. Cerezo, M., Verdon, G., Huang, H.-Y., Cincio, L. & Coles, P. J. Challenges and opportunities in quantum machine learning. *Nat. Comput. Sci.* **2**, 567–576, DOI: 10.1038/s43588-022-00311-3 (2022).

48. Biamonte, J. *et al.* Quantum machine learning. *Nature* **549**, 195–202, DOI: 10.1038/nature23474 (2017).

49. Cai, H., Ye, Q. & Deng, D.-L. Sample complexity of learning parametric quantum circuits. *Quantum Sci. Technol.* **7**, 025014, DOI: 10.1088/2058-9565/ac4f30 (2022).

50. Haug, T., Bharti, K. & Kim, M. S. Capacity and quantum geometry of parametrized quantum circuits. *PRX Quantum* **2**, 040309, DOI: 10.1103/PRXQuantum.2.040309 (2021).

51. Mari, A., Bromley, T. R., Izaac, J., Schuld, M. & Killoran, N. Transfer learning in hybrid classical-quantum neural networks. *Quantum* **4**, 340, DOI: 10.22331/q-2020-10-09-340 (2020).

52. Schetakis, N., Aghamalyan, D., Griffin, P. & Boguslavsky, M. Review of some existing qml frameworks and novel hybrid classical–quantum neural networks realizing binary classification for noisy datasets. *Sci. Reports* **12**, 11927, DOI: 10.1038/s41598-022-14876-6 (2022).

53. Schetakis, N. *et al.* Quantum machine learning for credit scoring. *Res. Collect. Sch. Comput. Inf. Syst.* 1–13 (2022).

54. Bharti, K. *et al.* Noisy intermediate-scale quantum algorithms. *Rev. Mod. Phys.* **94**, 015004, DOI: 10.1103/RevModPhys.94.015004 (2022).

55. Cerezo, M. *et al.* Variational quantum algorithms. *Nat. Rev. Phys.* **3**, 625–644, DOI: 10.1038/s42254-021-00348-9 (2021).

56. Schuld, M., Sweke, R. & Meyer, J. J. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Phys. Rev. A* **103**, 032430 (2021).

57. Bergholm, V. *et al.* Pennylane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968* (2018). [Online]. Available: http://arxiv.org/abs/1811.04968. Accessed: Jul. 31, 2023.

58. Bergmeir, C., Hyndman, R. J. & Koo, B. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Comput. Stat. & Data Analysis* **120**, 70–83, DOI: 10.1016/j.csda.2017.11.003 (2018).

59. Racine, J. Consistent cross-validatory model-selection for dependent data: Hv-block cross-validation. *J. Econom.* **99**, 39–61, DOI: 10.1016/S0304-4076(00)00030-0 (2000).

60. Zheng, W. Tscv: Time series cross-validation. *[Online]. Available: https://arxiv.org/abs/2307.02201* (2023).

61. Yu, Y., Hu, G., Liu, C., Xiong, J. & Wu, Z. Prediction of solar irradiance one hour ahead based on quantum long short-term memory network. *IEEE Transactions on Quantum Eng.* **4**, 1–15 (2023).

62. Sagingalieva, A. *et al.* Photovoltaic power forecasting using quantum machine learning. *arXiv preprint arXiv:2312.16379* (2023). [Online]. Available: http://arxiv.org/abs/2312.16379.

63. Sutskever, I. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems 27* (2014).

64. Schetakis, N. *et al.* Quantum machine learning for credit scoring. *Mathematics* **12**, 1391 (2024).

65. Scully, M. O. & Zubairy, M. S. *Quantum optics* (Cambridge university press, 1997).

66. Nielsen, M. A. & Chuang, I. L. *Quantum computation and quantum information* (Cambridge university press, 2010).

67. Bergholm, V. *et al.* Pennylane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968* (2018).

68. Larocca, M. *et al.* A review of barren plateaus in variational quantum computing. *arXiv preprint arXiv:2405.00781* (2024).

69. Barthe, A. & Pérez-Salinas, A. Gradients and frequency profiles of quantum re-uploading models. *Quantum* **8**, 1523 (2024).

70. Coelho, R., Sequeira, A. & Paulo Santos, L. Vqc-based reinforcement learning with data re-uploading: performance and trainability. *Quantum Mach. Intell.* **6**, 53 (2024).

71. Cerezo, M. & Coles, P. J. Higher order derivatives of quantum neural networks with barren plateaus. *Quantum Sci. Technol.* **6**, 035006 (2021).

72. Sen, P., Bhatia, A. S., Bhangu, K. S. & Elbeltagi, A. Variational quantum classifiers through the lens of the hessian. *Plos one* **17**, e0262346 (2022).

Acknowledgements

The authors would like to thank the European Union for funding this research through two projects: ERA4CH (Earthquake Risk Platform For European Cities Cultural Heritage Protection—grant agreement No. 101086280) and EYE (Economy bY spacE—grand agreement No. 10100763), both part of the Horizon 2020 research and innovation program. The research was also supported by the Cyprus National Project CODEVELOP-ICT-HEALTH/0322/0047.

Author contributions statement

The authors confirm contribution to the paper as follows: study conception and design: N.S., P.B., N.A., D.A., K.B., P.F., E.I.V.; data collection: P.F., E.I.V.; analysis and interpretation of results: P.B., N.S., K.B., N.A., D.A., S.I.T., A. A., E.I.V.; draft manuscript preparation: N.S., P.B., N.A., P.F.; All authors reviewed the manuscript.